Habib University
shaping futures

CS201 – Data Structures II

Lecture 06

Dr. Muhammad Mobeen Movania

Outline

- What is hashing
- What are hashed list searches
- Hashing methods
  - Direct
  - Subtraction
  - Modulo division
  - Digit extraction
  - Mid square
  - Folding
  - Rotation
  - Pseudorandom
- Spatial hashing
- Collision Resolution
- Collision Resolution Methods

Hashed List Searches

- List searches require a lot of tests before the data is found
- Ideal case: We know where our data is and we may access it directly
- Goal of Hashed Search: Find data in O(1)

What about Map ADT ?

- A Map stores “key” “value” pairs (k,v) called entries
- Each key should be unique
- There is a mapping from k to v
- k can be a primitive data type or an object
- It can be same as a part of the value
- E.g. phone book, yellow pages, dictionary
- If key is a non-integral type, an integer hash code is associated with the key

- Question:
  - How are they implemented? and Why are they fast?
- Answer:
  - Hashing

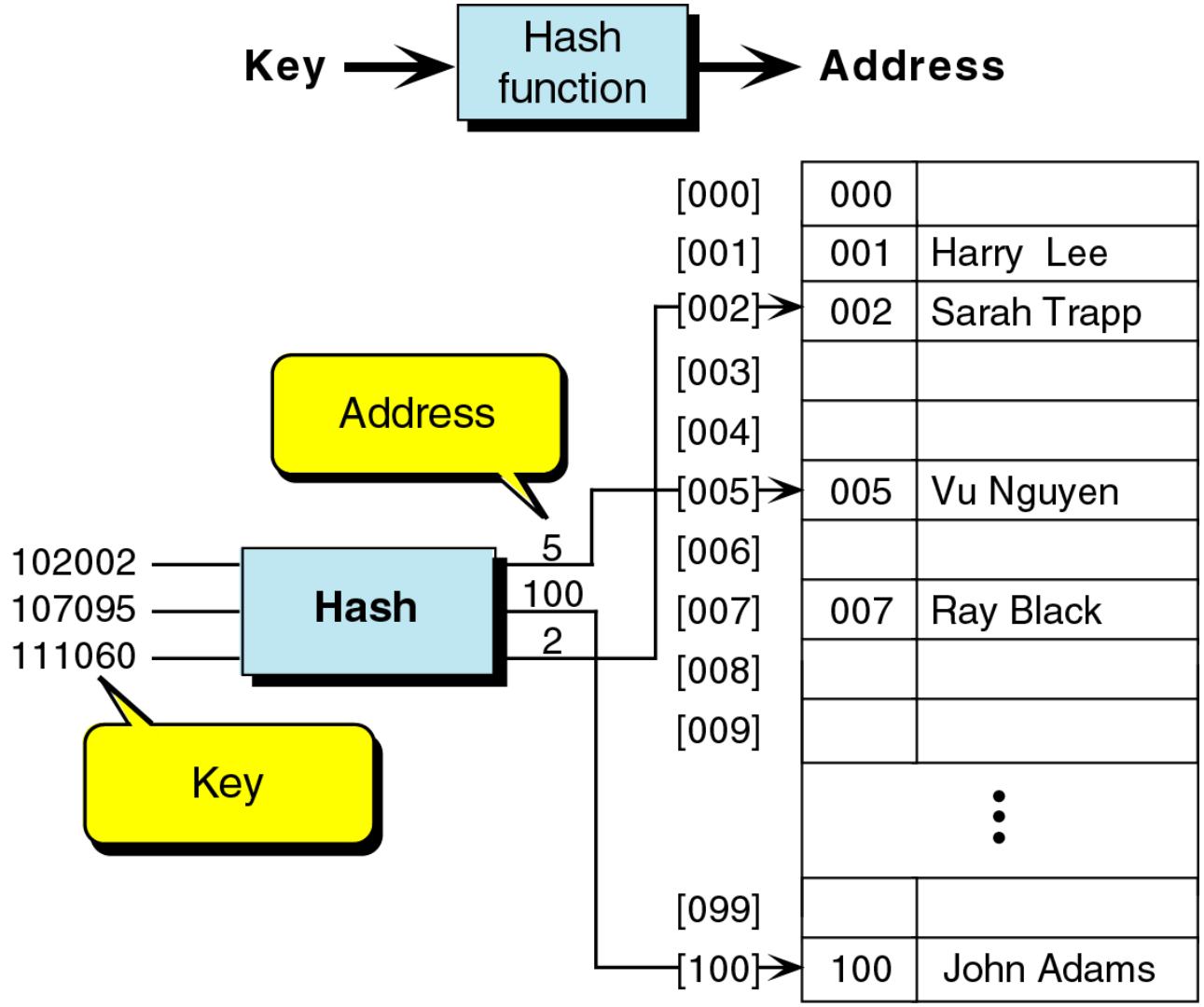
What is Hashing?

- Generating address from a key
- Key is a record's property on which a record is stored in memory (called **prime area**)
- A hash search is a search in which the key is modified (using an algorithm) into a memory address (called **home address**)
- The conversion process of **converting a key to a memory address** is called **Hashing**

Hashing Terms

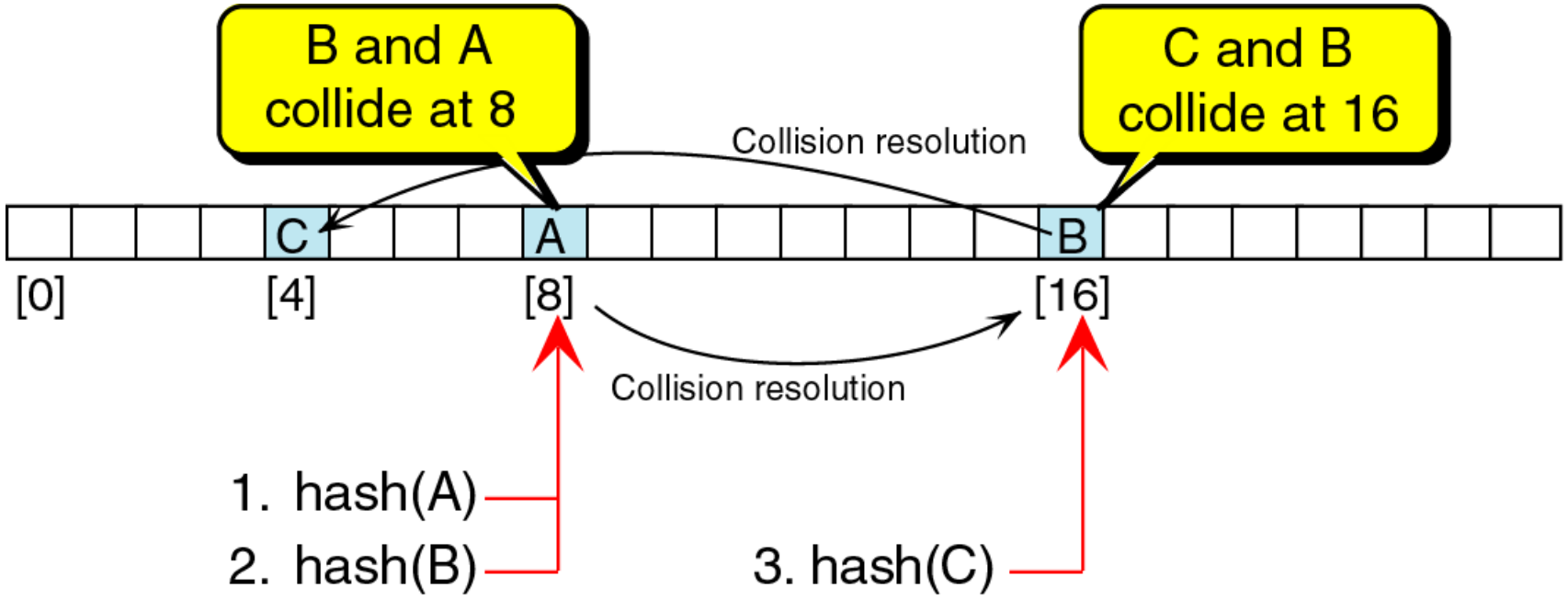
- Usually there are more keys than can be accommodated in memory
- More than one keys that hash to the same location are called **synonyms**
- A hash **collision** occurs when an key is inserted into an occupied memory location
- Calculation of a memory address and its test for collision is called a **probe**

Hashing

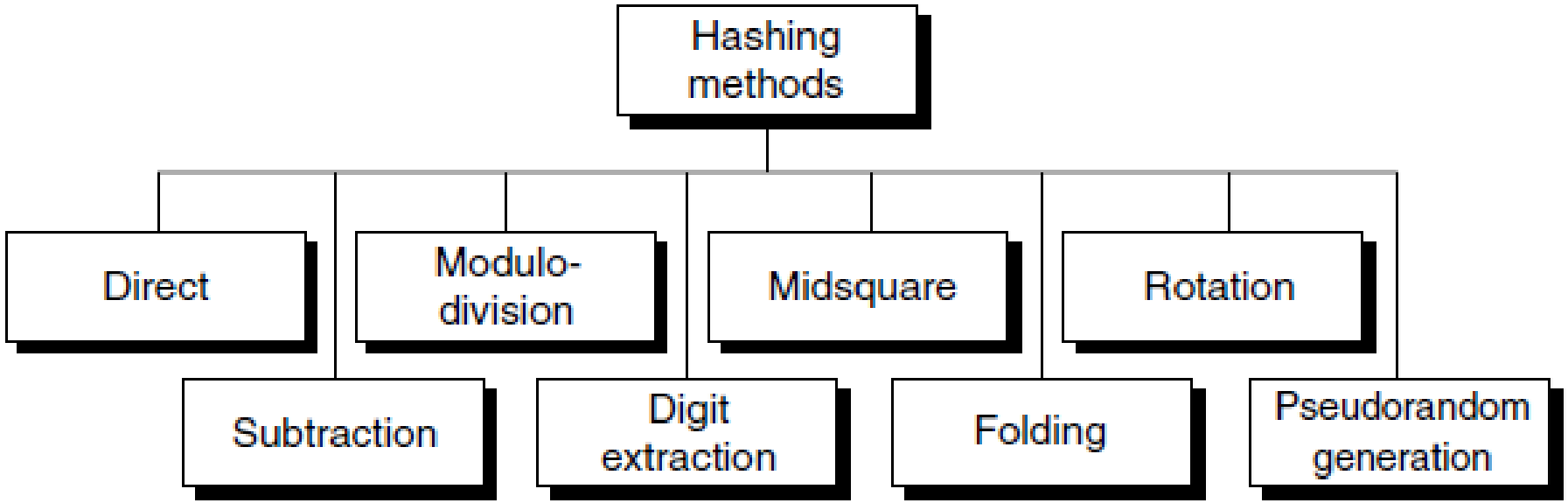


Visualization:
https://www.cs.usfca.edu/~galles/visualization/OpenHash.htm

Hash Collision

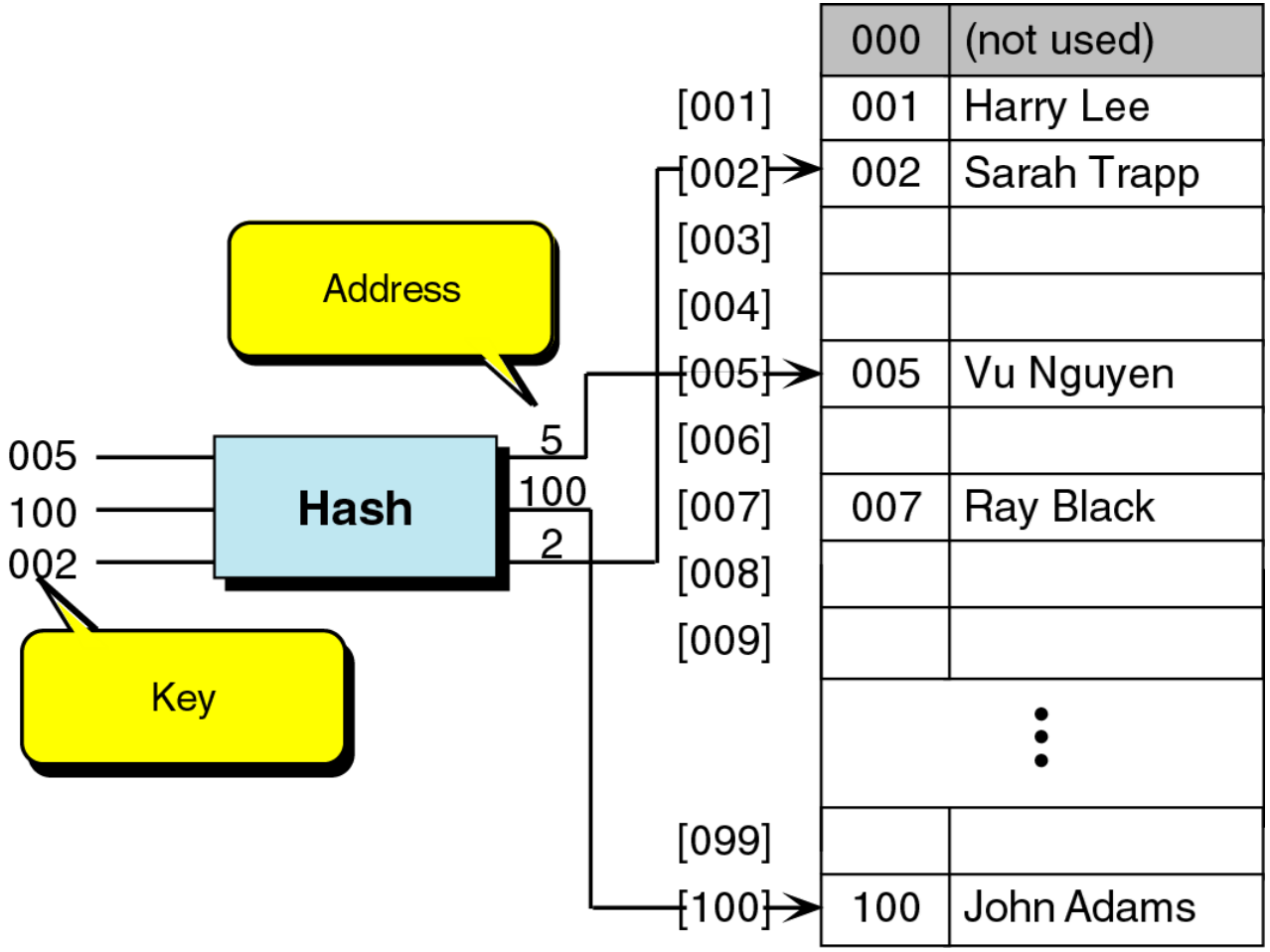


Hashing Methods



Direct Method

- The key is the address without any algorithmic modification



Subtraction Method

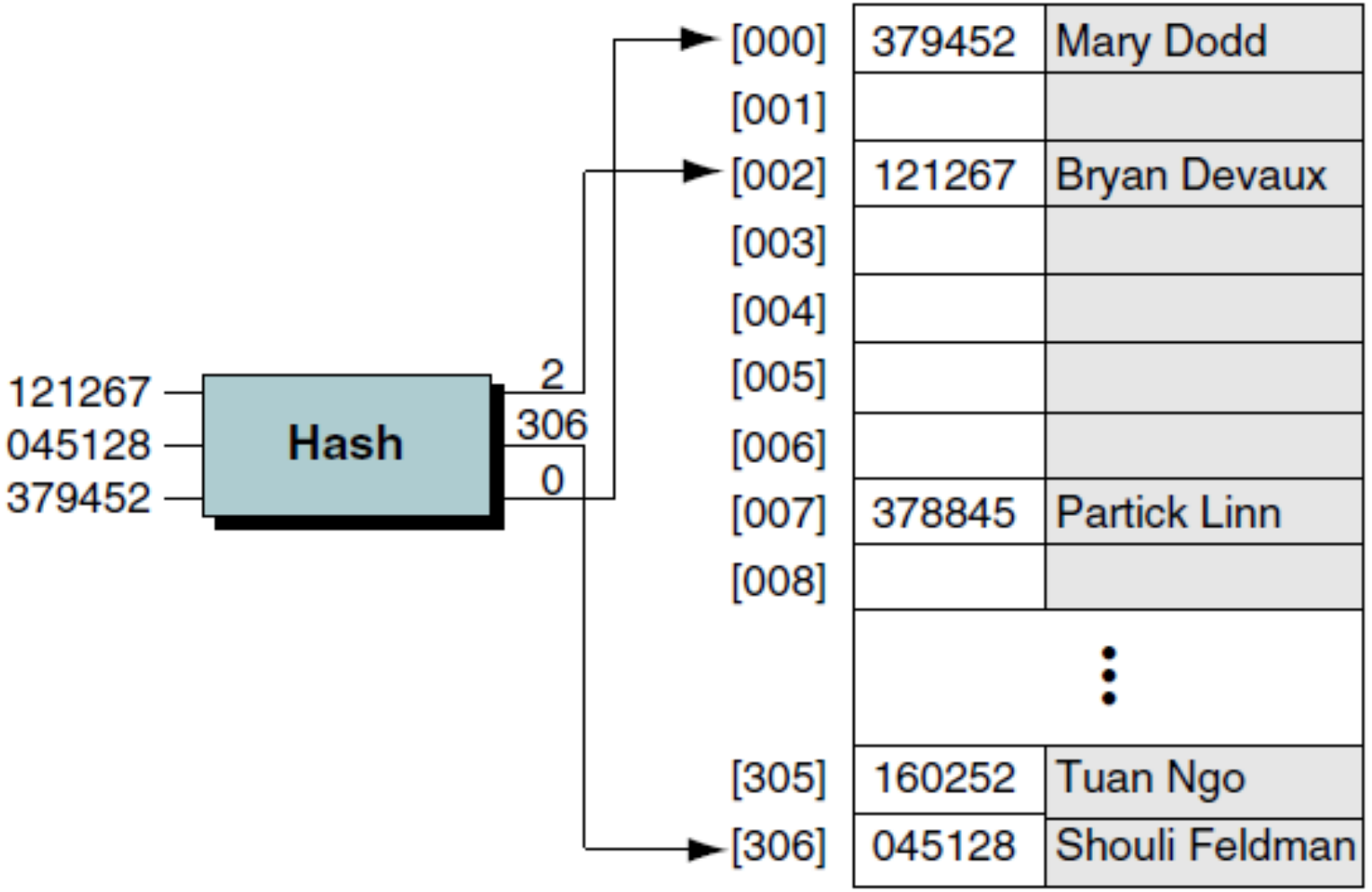
- If keys do not start from 0 or 1, you get an index by subtracting a constant value from the key

- The direct and subtraction hash functions both **guarantee a search** effort of one **with no collisions**.
  - They are one-to-one hashing methods that is only one key hashes to each address.

Modulo Division Method

- Also known as division remainder or modulo-division method
- Divides the key by list/array size and uses the remainder as index
- Works with any size but prime numbers give less collisions

Modulo Division Example



Modulo Division Example

- Assume array size of 307
- Lets consider the following keys 121267,045128 and 379452

Input	Hash Calculation	Index/Address
121267	121267 % 307	2
045128	045128 % 307	306
379452	379452 % 307	0

Digit Extraction Method

- Extract certain digits and then use the obtained number as address
- Assuming that we have an array size of 1000 (indices from 0 to 999) we can use three digits
- Example: (extracting 1st , 3rd and 4th digit)
  - **1**4**56**74 => 156
  - **3**4**45**65 => 345
  - **1**3**25**54 => 125

Midsquare Method

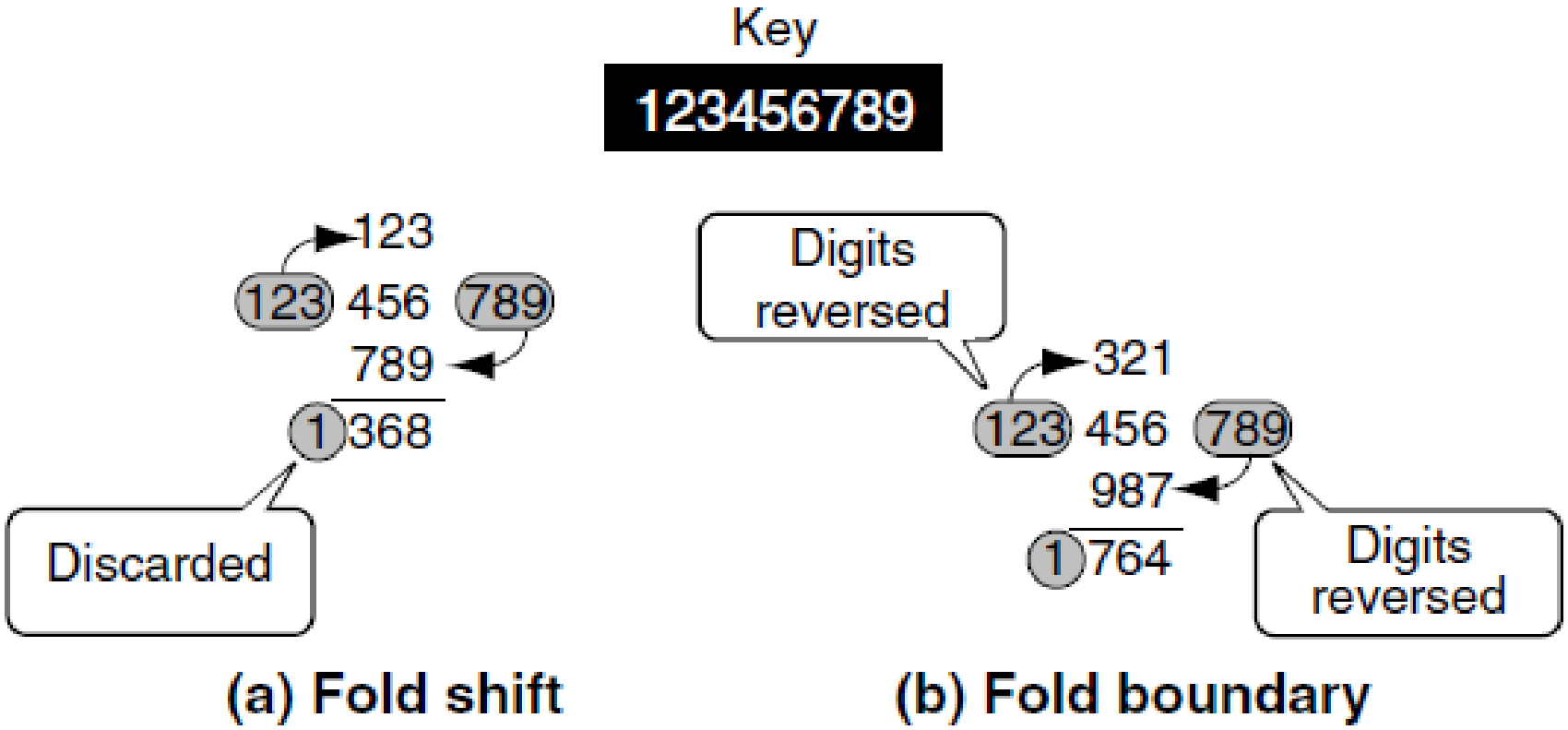
- The key or a portion of it is squared and the address is selected from the mid of the squared number
- Example:
  - Assuming we have a three digit key and three digit address

Input	Square	Index/Address
121	1**464**1	464
365	13**322**5	322
749	56**100**1	100

Folding Method

- Two methods
  - Fold shift
  - Fold boundary
- Fold shift
  - Key is divided into parts whose size matches the required address size
  - The left and right parts are shifted and added to the middle part
- Fold boundary
  - The key is divided into parts as in fold shift
  - The digits of left and right parts are reversed and then added to the middle part
- In both folding methods, overflowing digits are discarded

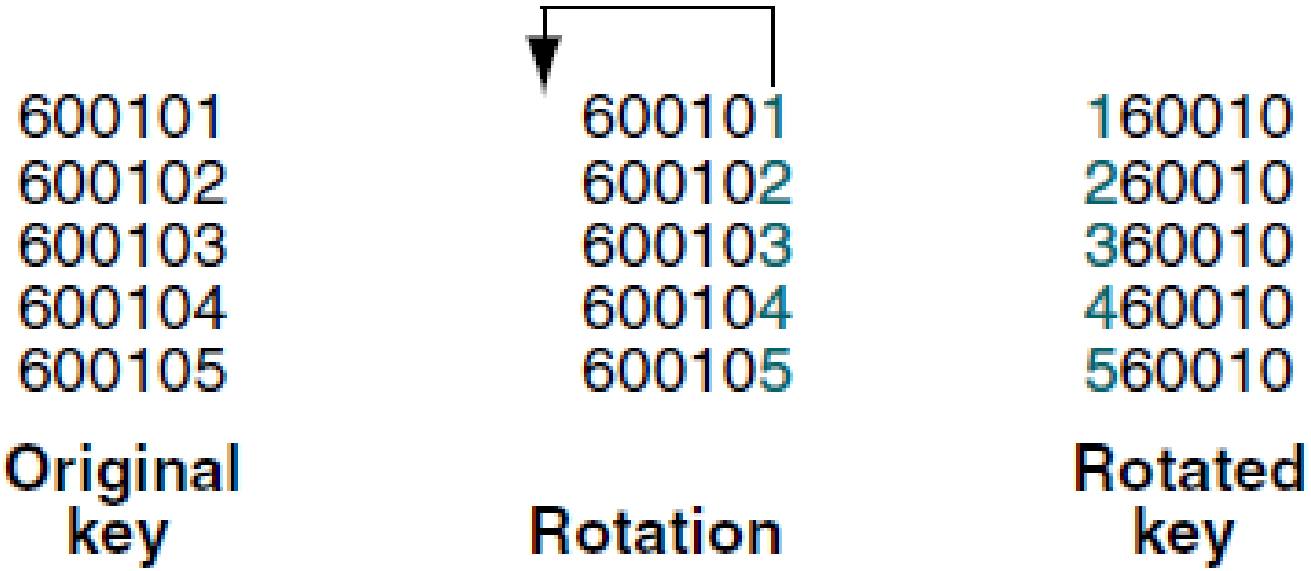
Folding Example



Rotation Method

- Generally used in conjunction with other methods
- Useful when keys are assigned serially as in employee ids and part numbers etc.
- Such numbers usually end up with identical numbers that differ by 1 digit only
- When these numbers are used in hashing, they tend to hash to the same address
- To solve this, the last digits are rotated to the front of the key
- Usually performed by bitwise shift (<<, >>) and bitwise and (&) operator

Rotation Example



Pseudorandom Method

- Key is used as a seed in a pseudorandom number generator
- The resulting random number is then scaled to the range of addresses using modulo division

$$y = ax + c$$

  - Where,
    - x is the key
    - a is a scaling coefficient
    - c is a constant
- The remainder of the result (y) is then taken with the list/array size

$$index = y \,\%\, size$$

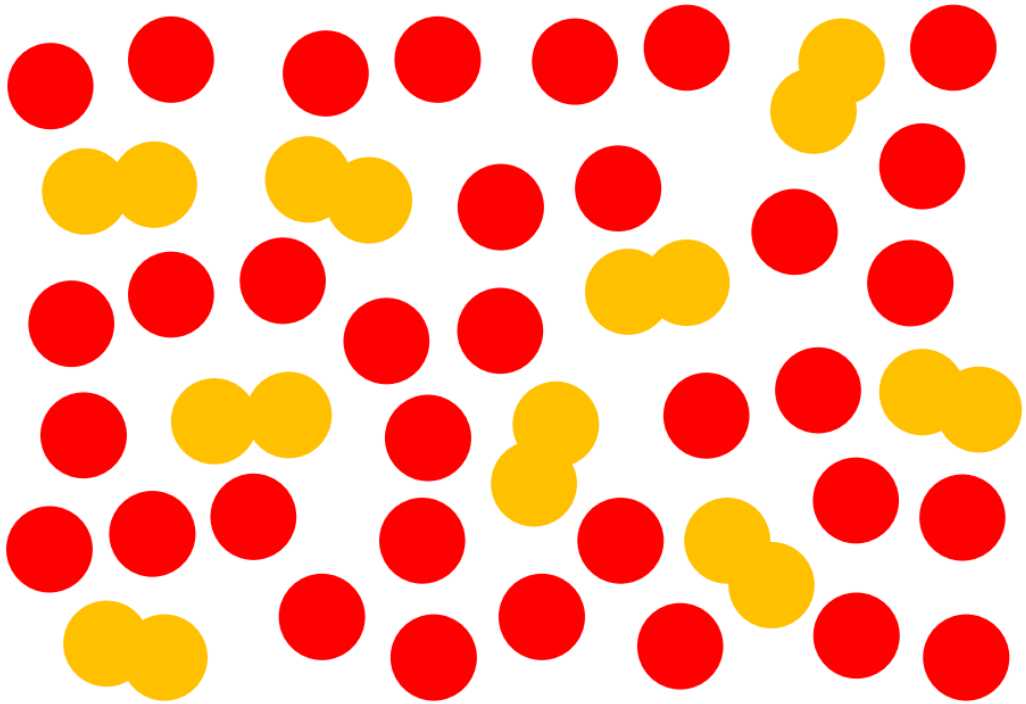
Pseudorandom Method

- Example

```
y = ((17 * 121267) + 7) modulo 307
y = (2061539 + 7) modulo 307
y = 2061546 modulo 307
y = 41
```

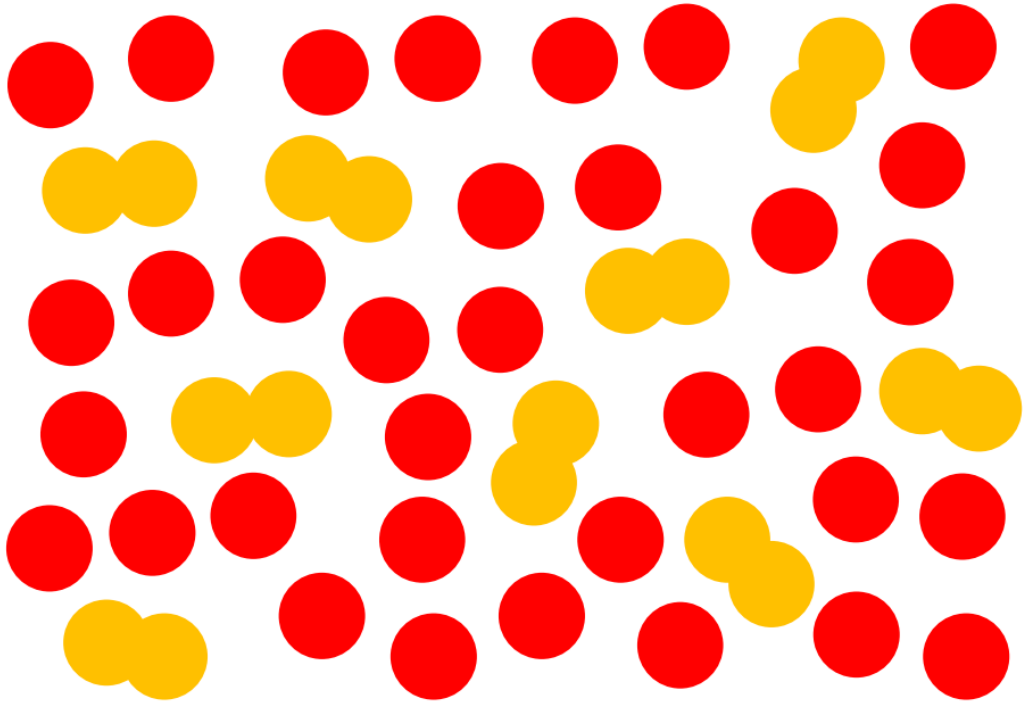
Spatial Hashing

- Problem:
  - Given n points, find all collisions of points with each other



Spatial Hashing

- Solution:
  - For all points $\mathbf{p}_i$: Find neighbors $\mathbf{p}_j$ such that $|p_i - p_j| \leq d$
  - Special case $d = 2r$: Find overlaps of particles with radius *r*



Spatial Hashing

- Brute force solution:

for all points $\mathbf{p}_i$
  for all points $\mathbf{p}_j$
    if $|\mathbf{p}_j - \mathbf{p}_i| \leq d$
      handleOverlap(i, j)

- Problems:
  - This is $O(n^2)$, for 100,000 points (common) we perform 10,000,000,000 test

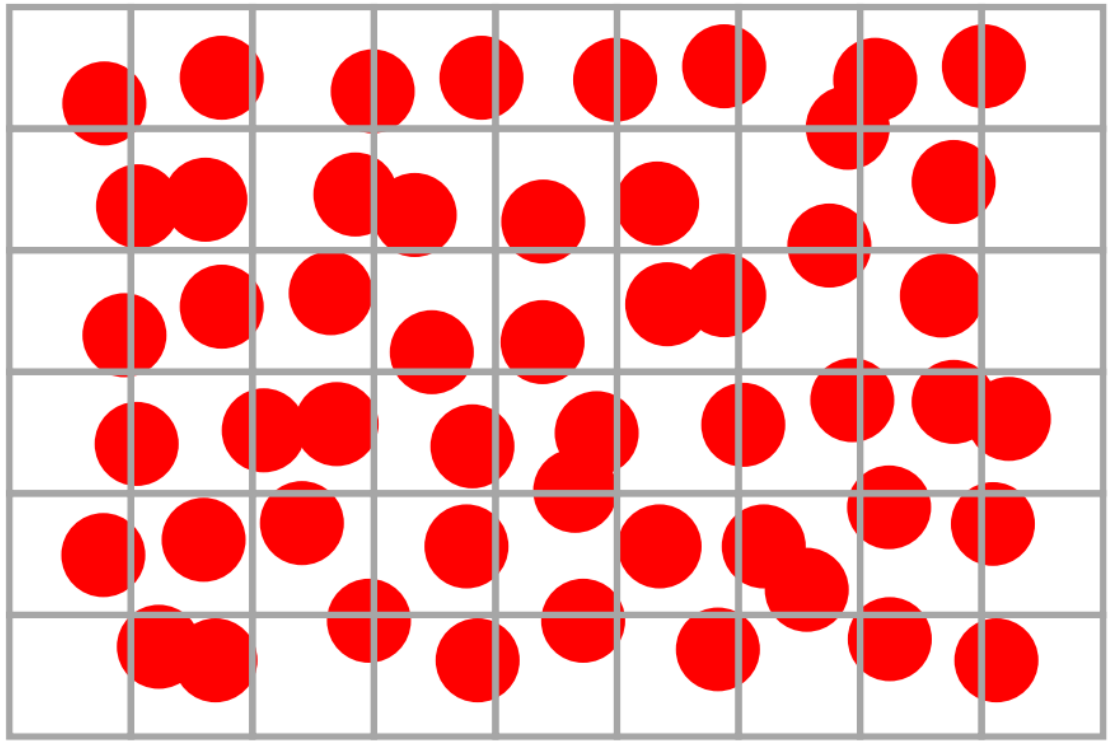
Spatial Hashing

- In general for the complexity of a simulation algorithm:
  - $O(n)$ is good
  - $O(n \log n)$ is OK (i.e. sorting, log 100,000 ≈ 17)
  - $O(n^2)$ is not an option

- What should we do then?
  - Think of how we can use some sort of acceleration scheme

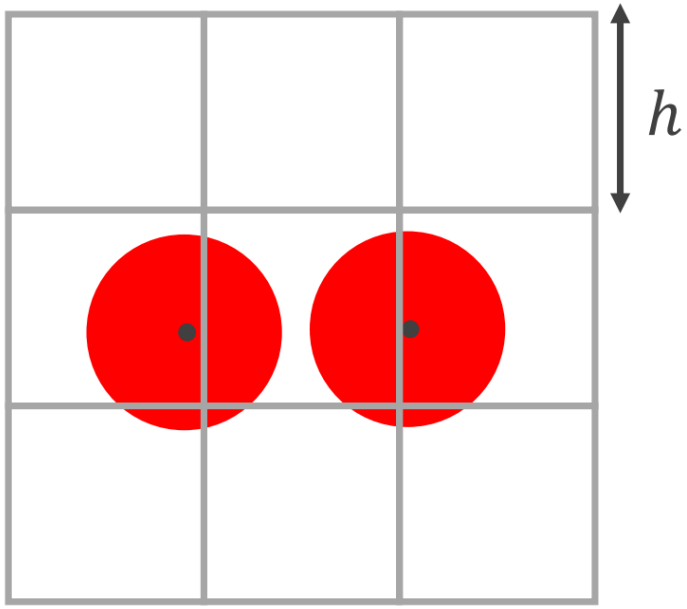
Spatial Hashing

- Key Idea:
  - Store particles in a regular grid
  - Instead of checking with all other particles, only check particles in the same and surrounding cells

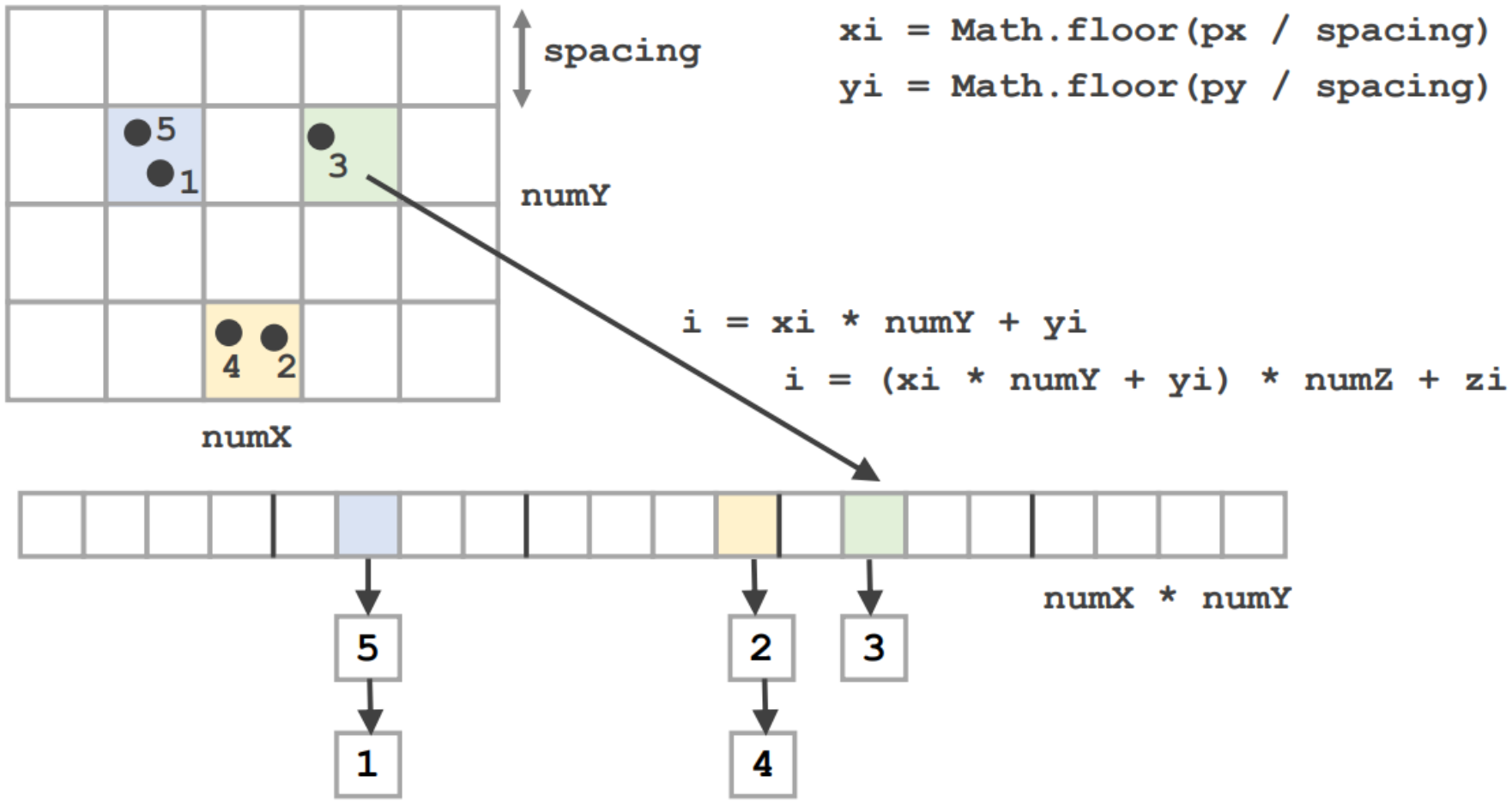


Spatial Hashing

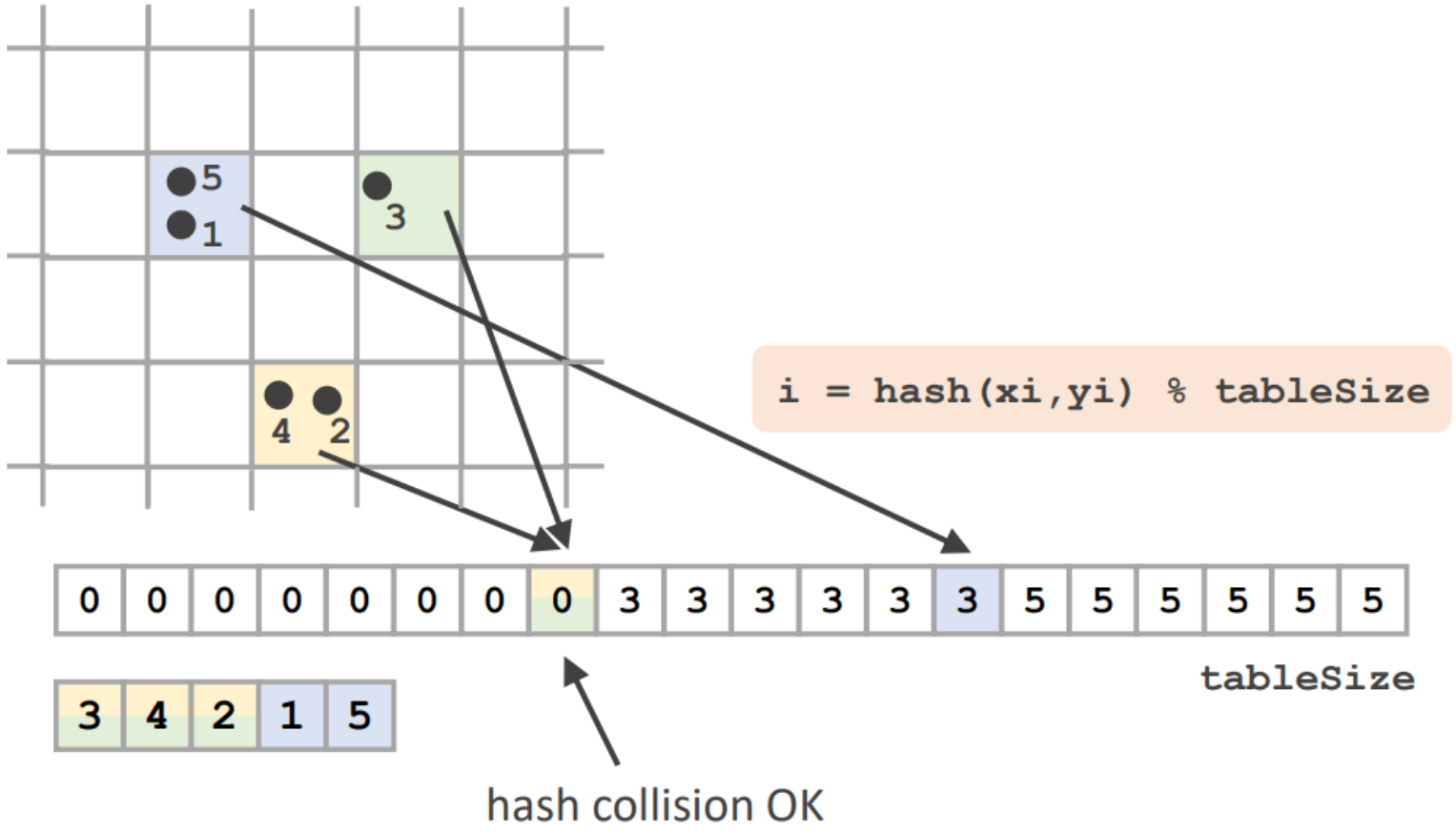
- Grid layout
  - Store particles once where their center is locate
  - If we choose h = 2r, we only need to check the containing and the surrounding cells
  - 9 in 2 dimensions, 27 in 3 dimensions



Storing the Grid



Handle unbounded grid using Spatial Hashing



Hash function

- The hash function must return a value that distributes the cells evenly

```
hashCoords(xi, yi, zi) {
    var h = (xi * 92837111) ^ (yi * 689287499) ^ (zi * 283923481);
    return Math.abs(h) % this.numCells;
}
```

- What should be the hash table size
  - Large hash table size: fewer collisions, fewer tests, faster
  - Choosing tableSize = numParticles

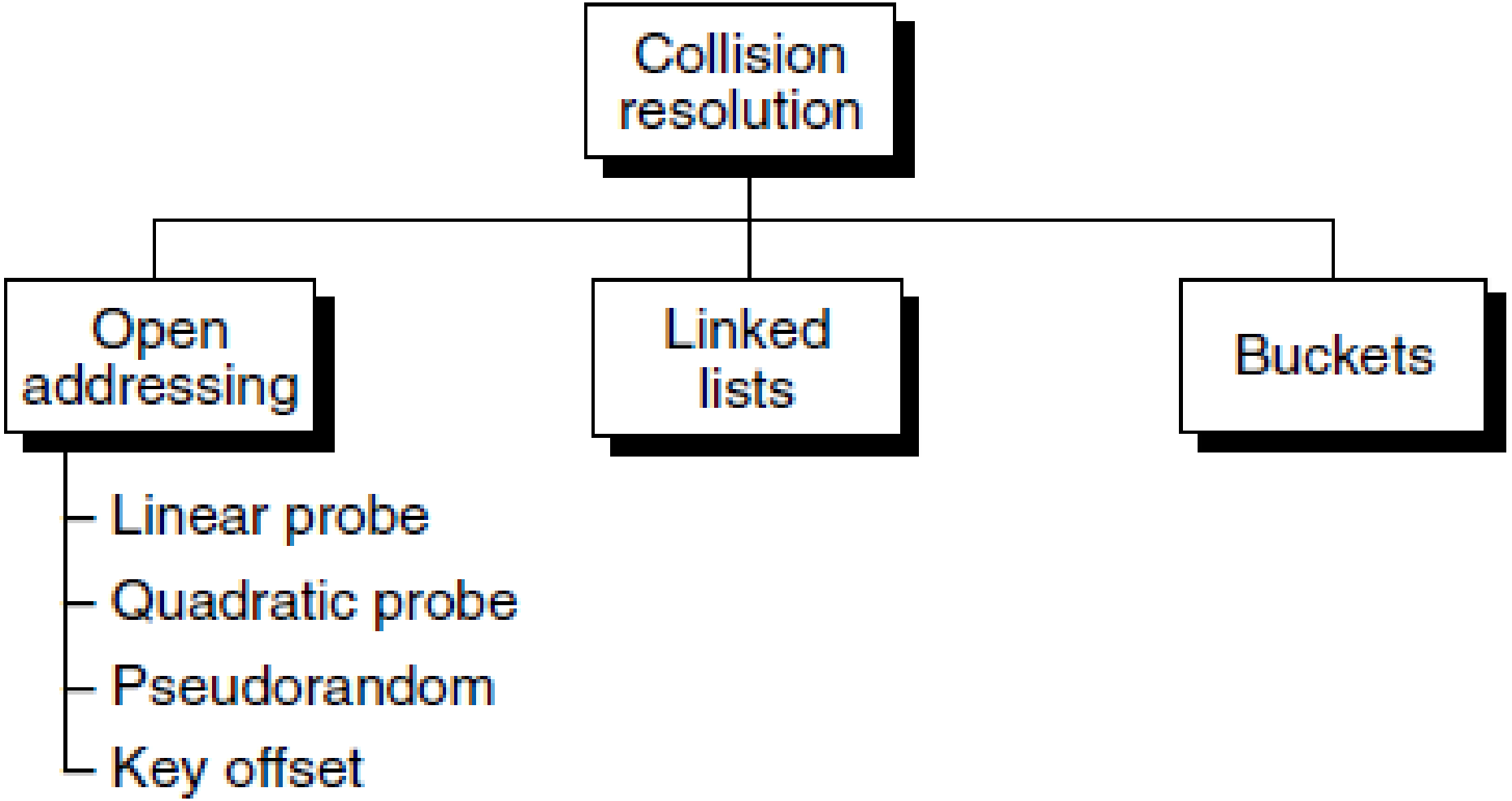
Hash Collision Resolution

- Apart from direct and subtraction methods, all other hashing methods suffer from collisions
- Some hashed list terms:
  - Load factor ($\propto$): Percentage of the number of elements in list divided by the total list size

$$\propto = \frac{k}{n} * 100$$

  - Clustering: Clustering is a build up of data unevenly across the hashed list due to collision resolution.
    - Primary Clustering: When clustering is at home address
    - Secondary Clustering: When clustering is at collision path

Collision Resolution Methods



Open Addressing

- Resolves collision in the prime area (where all home address are stored)
- When collision occurs, prime area is searched for empty cells
- Three types
  - Linear probe
  - Quadratic probe
  - Double hashing
    - Pseudorandom rehashing
    - Key offset rehashing

Linear Probe

- In case of collision, add 1 to the current address/index
- If the new address is occupied add 1 again to find an empty address
- An alternate scheme adds 1, if collision is found, it subtracts 2, if another collision is found, it adds 3 and so on
- Generated address must be within the address range
- Advantages
  - Simple to implement
  - Data remains near home address
- Disadvantages
  - Creates primary clusters
  - Makes search algorithm complex due to addition and deletion

Linear Probing Visualization

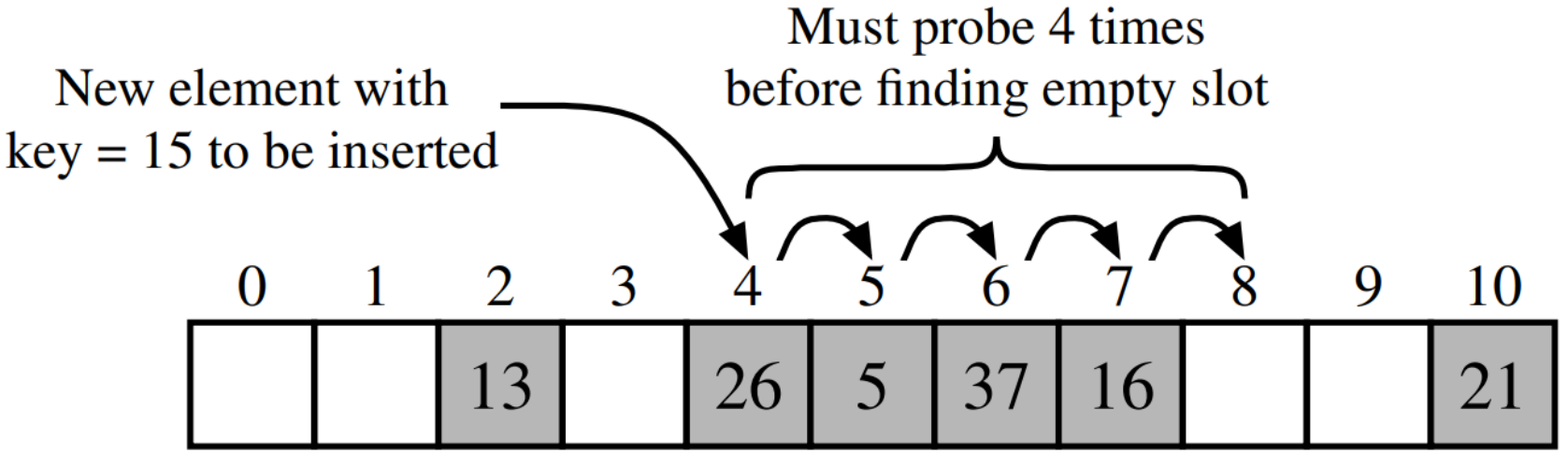


Figure 10.7: Insertion into a hash table with integer keys using linear probing. The hash function is $h(k) = k \bmod 11$. Values associated with keys are not shown.

Example – Linear Probing

- Elements: {14, 82, 44, 3, 89, 76, 4, 64, 84, 45}
- Size = 20
- Hash function: h(x) = x mod size
- In case of collision, add 1 until you find a free slot
  - Collisions are shown in red

Index	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19
Linear Probing			82	3	44	4	64	84	45	89					14		76			

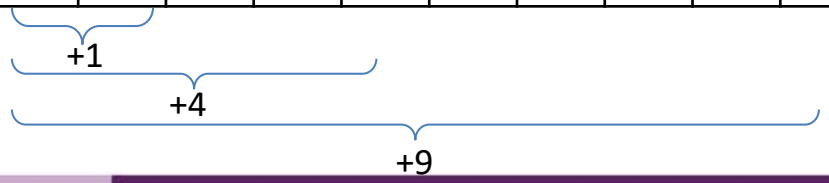
Quadratic Probe

- Instead of adding 1, we add the collision probe number squared
  - For 1 collision probe, add 1^2=1, for 2 collision probe, add 2^2=4, for 3 collision probe, add 3^2=9 and so on
  - Continued until we find an empty address or we don't have enough elements
- Advantages
  - Easy to implement
  - Avoids primary ~~and secondary~~ clustering
- Disadvantages
  - More time required for squaring the probe number which can be avoided by just doing a scalar multiplication
  - It is not possible to generate a new address for every element

Example – Quadratic Probing

- Elements: {14, 82, 44, 3, 89, 76, 4, 64, 84, 45}
- Size = 20
- Hash function: h(x) = x mod size
- In case of collision,
  - add 1^2 if one collision probe used, 2^2 if two collision probes used, y^2 if y collision probes used
  - Collisions are shown in red

Index	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19
Linear Probing			82	3	44	4	64	84	45	89					14		76			
Quadratic Probing			82	3	44	4	45		64	89				84	14		76			



Double Hashing Collision Resolution

- In double hashing, rather than using arithmetic, the address is rehashed
- Psuedorandom Rehashing
  - We use a pseudorandom number generator to rehash the address
- Keyoffset Rehashing
  - We use an offset to rehash the address
- Advantages
  - Prevents primary clustering

Linked List Resolution (Chained Hashing)

- General disadvantage of open addressing is that each collision resolution increases chances of future collision
- In linked list resolution, a separate area (called overflow area) is used for collisions
  - All synonyms are chained into a linked list and are then attached to the prime area using a pointer
  - In case of collision, one element is stored in prime area and it is chained to its corresponding linked list in the overflow area through a linked list pointer

Chained Hashing Visualization

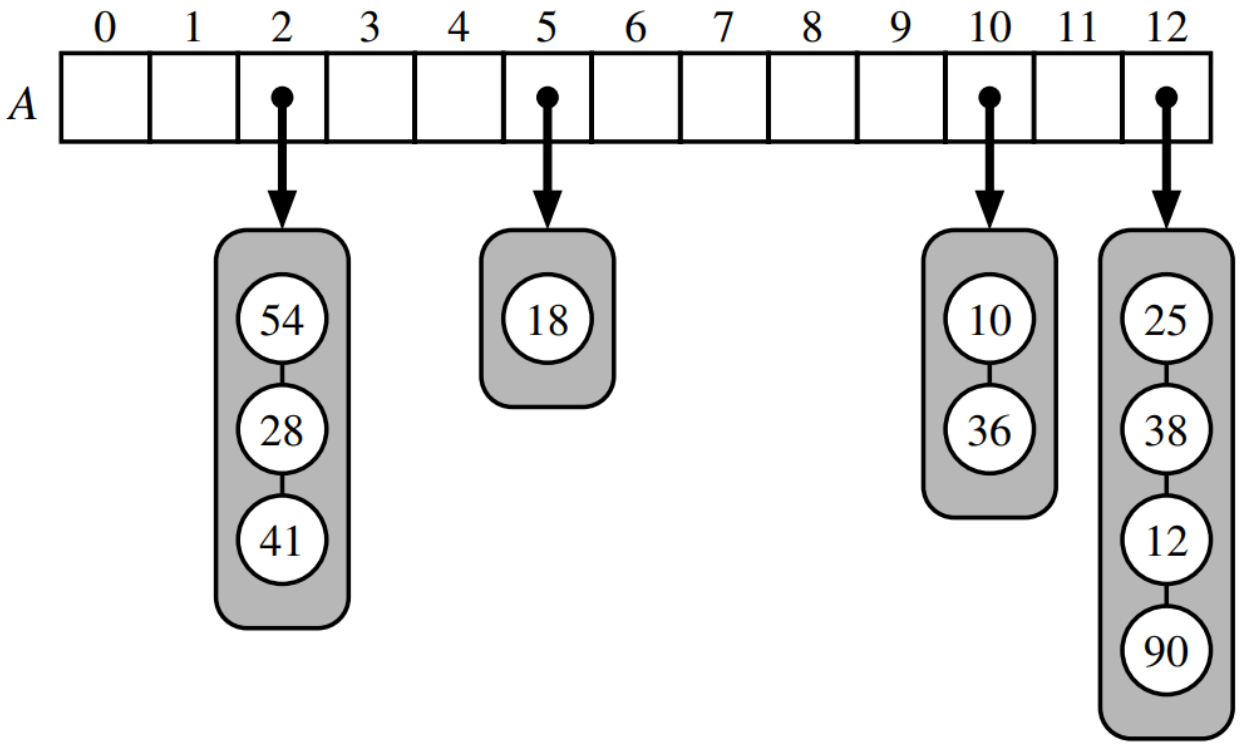


Figure 10.6: A hash table of size 13, storing 10 items with integer keys, with collisions resolved by separate chaining. The compression function is $h(k) = k \bmod 13$. For simplicity, we do not show the values associated with the keys.

Chained Hashtable

```
class ChainedHashTable {
    array<List> t;
    int n;
};
```

- Stores an array of List and total number of items n in the hash table
  - Given data item x is hashed using hash function hash(x)
  - The hash values are from {0,1,2,…, t.length}

- Invariant: n <= t.length
- Average number of elements stored in one of these lists is n/t.length ≤ 1.

What should be the hash function?

- Performance of a hash table depends critically on the choice of the hash function
- A good hash function
  - will spread the elements evenly among the t.length lists, so that the expected size of the list t[hash(x)] is O(n/t.length) = O(1)
- A bad hash function will
  - hash all values (including x) to the same table location, in which case the size of the list t[hash(x)] will be n so complexity will be O(n)

- Key properties of hash function
  - Uniformly distributed
  - Low probability of collision
  - Computationally fast

Example: Multiplicative Hashing

$$hash(x) = ((z \cdot x) \bmod 2^w) \text{ div } 2^{w-d}$$

Where,

x is a integer in $\{0,...,2^{w-1}\}$

z is a randomly chosen odd integer in $\{1,...,2^{w-1}\}$

w is the number of bits in the integer

Two parts of this hash function

- mod with 2^w part determines the state of the least significant w-1 bits: $x \% 2^r = x \,\&\, (2^r - 1)$
- div with 2^{w-d} drops the least significant w-d bits meaning it takes d significant bits

```
int hash(T x) {
   return ((unsigned)(z * hashCode(x))) >> (w-d);
}
```

Example:

- Given data (w=32, d=8)
  - 2^w=4294967296
  - z=4102541685
  - x=42
  - 2^w-1=4294967295

```
1 00000000 00000000 00000000 00000000
  11110100 10000111 11010001 01110101
  00000000 00000000 00000000 00101010
  11111111 11111111 11111111 11111111
```

- z.x
- (z.x) mod 2^w
- ((z.x) mod 2^w) div 2^{w-d}

```
101000 00011110 01001000 01011101 00110010
       00011110 01001000 01011101 00110010
                                  00011110
```

Performance of ChainedHashtable

Theorem 5.1. *A* `ChainedHashTable` *implements the* `USet` *interface. Ignoring the cost of calls to* `grow()`*, a* `ChainedHashTable` *supports the operations* `add(x)`, `remove(x)`, *and* `find(x)` *in* $O(1)$ *expected time per operation.*

- If considering the cost of grow
  - beginning with an empty ChainedHashTable, any sequence of m add(x) and remove(x) operations results in a total of O(m) time spent during all calls to grow().
  - the analysis is exactly like we did for ArrayList resize function earlier

Bucket Hashing

- Buckets can contain multiple data occurrences
- Advantages
  - Can store multiple data items during collision
- Disadvantages
  - Takes significantly more space as a lot of buckets remain empty or partially empty
  - The memory is fragmented considerably due to partially empty buckets
  - It cannot completely resolve collisions

Bucket Hashing Visualization

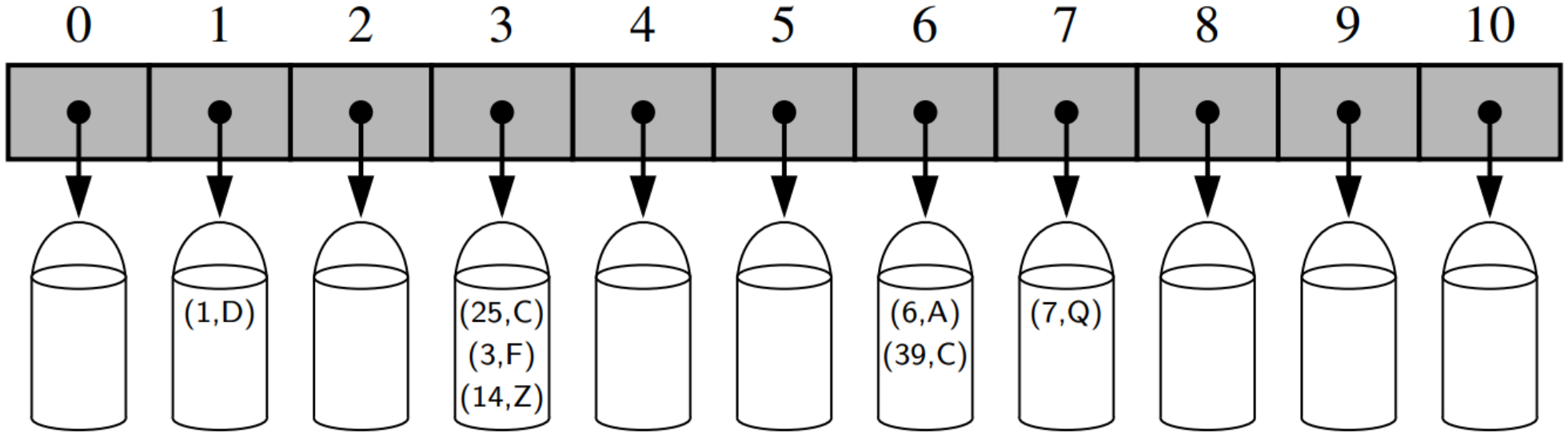


Figure 10.4: A bucket array of capacity 11 with items (1,D), (25,C), (3,F), (14,Z), (6,A), (39,C), and (7,Q), using a simple hash function.

Time Complexity

Operation	List	Hash Table	
		expected	worst case
__getitem__	$O(n)$	$O(1)$	$O(n)$
__setitem__	$O(n)$	$O(1)$	$O(n)$
__delitem__	$O(n)$	$O(1)$	$O(n)$
__len__	$O(1)$	$O(1)$	$O(1)$
__iter__	$O(n)$	$O(n)$	$O(n)$

Table 10.2: Comparison of the running times of the methods of a map realized by means of an unsorted list (as in Section 10.1.5) or a hash table. We let n denote the number of items in the map, and we assume that the bucket array supporting the hash table is maintained such that its capacity is proportional to the number of items in the map.

Book Reading

- ODS: Chapter 5, pages 107-122
- Goodrich: Chapter 10, section 10.2 pages 410-426

Next Lecture

- Lecture 08